

**UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJE Y COMPILADORES (2026-I)**

**INFORME DE INVESTIGACIÓN: ESTUDIO COMPARATIVO
DE PARADIGMAS, LENGUAJES Y DISEÑO DE DSL**

Asignación II: Los Lenguajes de Programación

Autores:

Fernando Centeno (C.I. 30.810.484)
Juan Longart (C.I. 31.882.343)
Adrian Reina (C.I. 31.317.970)
Rafael Rodríguez (C.I. 31.882.367)

Profesor:

Félix Márquez
fmarquez@e.uneg.edu.ve

Ciudad Guayana, Mayo de 2026

1. Introducción

En el desarrollo de software y en las ciencias de la computación, el estudiante comúnmente interactúa con los lenguajes de programación como un usuario final o programador aplicativo. En este nivel inicial, el enfoque principal radica en traducir lógica pura a una sintaxis particular con el fin de resolver problemas de complejidad moderada. Sin embargo, al alcanzar el nivel del diseño de compiladores e intérpretes, se vuelve indispensable contemplar el lenguaje no como una herramienta inalterable, sino como un producto de software minuciosamente diseñado, provisto de una arquitectura lingüística específica y sujeto a restricciones formales rigurosas.

Las decisiones de diseño léxico, morfológico y sintáctico tomadas por los diseñadores de lenguajes impactan directamente la eficiencia de la ejecución, la expresividad del código, el consumo de memoria y la mantenibilidad a largo plazo.

Este informe presenta un estudio comparativo riguroso que abarca:

1. Un análisis crítico de los paradigmas fundamentales y emergentes de la programación.
2. Un análisis morfológico y sintáctico a bajo nivel de cuatro lenguajes representativos (Zig, Python, Rust y JavaScript) respaldado por pruebas de rendimiento (benchmarking) en condiciones controladas.
3. El diseño formal de un Lenguaje de Dominio Específico (DSL) llamado Lenguaje L, orientado a la gestión del sistema crítico de microredes eléctricas inteligente y almacenamiento de energía ECO-GRID.

2. Actividad I: Matriz Descriptiva y Análisis de Paradigmas

El desarrollo de software contemporáneo está marcado por el auge del desarrollo multiparadigma, donde lenguajes de propósito general (como C++, Rust, Python o C#) incorporan características de múltiples filosofías de diseño para responder a los retos del mercado. A continuación, se definen los ejes temáticos y operativos de los principales paradigmas:

2.1. Paradigma Imperativo/Estructural

* **Gestión Explícita del Estado:** Se basa en la modificación del estado del sistema mediante sentencias imperativas secuenciales. La máquina sigue un orden de ejecución estricto donde el estado global o local se altera continuamente.

* **Secuenciación de Instrucciones:** El flujo de control se define explícitamente mediante estructuras de secuencia, selección (if/else) e iteración (bucles).

* **Mutabilidad de Memoria y Efectos Secundarios:** Las variables representan celdas físicas de memoria que pueden ser sobrescritas libremente. Esto permite algoritmos de alta velocidad y bajo consumo de memoria, pero dificulta el análisis estático y la depuración del código debido a efectos secundarios imprevistos donde una función altera variables compartidas globales o fuera de su alcance local.

2.2. Paradigma Orientado a Objetos (POO)

* **Encapsulamiento:** Agrupa datos (atributos) y comportamiento (métodos) en una única unidad lógica llamada clase u objeto, restringiendo el acceso directo a los detalles de implementación interna.

* **Polimorfismo:** Capacidad de una referencia de objeto para comportarse de distintas maneras según el tipo de dato real asignado en tiempo de ejecución.

* **Herencia vs. Composición:** La herencia modela relaciones "es un" mediante la jerarquía de clases compartiendo atributos y métodos. La composición, modelando relaciones "tiene un", prioriza acoplar objetos independientes para construir comportamientos más complejos, siendo esta última la estrategia recomendada en el diseño de software moderno (ej. Rust favorece la composición a través de rasgos/traits).

* **Abstracción Basada en Datos y Comportamiento:** Permite a los ingenieros modelar el mundo real en entidades de software cohesivas y reutilizables.

2.3. Paradigma Funcional

* **Inmutabilidad de Datos:** Una vez que se crea un valor, este no puede ser modificado. Las transformaciones producen nuevos valores en lugar de mutar los existentes.

* **Funciones como Ciudadanos de Primer Orden:** Las funciones pueden ser asignadas a variables, pasadas como argumentos a otras funciones y retornadas como resultados.

* **Evaluación Perezosa (Lazy Evaluation):** El cómputo de una expresión se posterga hasta que su valor sea estrictamente requerido para la ejecución, lo que permite manejar estructuras de datos potencialmente infinitas.

* **Transparencia Referencial y Eliminación de Efectos Colaterales:** Una función pura siempre produce el mismo resultado para los mismos argumentos, careciendo de interacciones externas que alteren el sistema. Esto simplifica drásticamente el razonamiento matemático del programa y facilita la ejecución concurrente.

2.4. Paradigma Lógico/Declarativo

* **Programación Basada en Relaciones:** En lugar de especificar el "cómo" resolver un problema, el programador define el "qué" mediante hechos y reglas lógicas.

* **Unificación y Resolución de Cláusulas de Horn:** El motor de inferencia busca emparejar variables utilizando reglas de unificación, resolviendo metas a partir de cláusulas de Horn y realizando backtracking automático cuando una ruta falla.

* **Abstracción Total del Flujo de Control:** El programador carece del control directo sobre la secuencia temporal de las instrucciones; el motor deductivo del compilador/intérprete gestiona la búsqueda del espacio de soluciones.

2.5. Paradigma Concurrente/Actores (Emergente)

* **Modelos de Paso de Mensajes:** Los procesos no comparten variables en memoria común. La comunicación se realiza exclusivamente enviando mensajes asíncronos a las colas ("buzones") de otros actores.

* **Aislamiento Estricto de Estado:** Cada actor gestiona su propio estado interno de forma totalmente aislada. Ningún otro actor puede leer o escribir directamente en su memoria.

* **Mitigación de Condiciones de Carrera:** Al eliminar la memoria mutable compartida, se eliminan nativamente las condiciones de carrera y los interbloqueos (deadlocks), resolviendo la complejidad inherente al paralelismo y la concurrencia a nivel de diseño lingüístico.

3. Actividad II: Estudio Comparativo de Lenguajes y Benchmarking

3.1. Análisis Morfológico (Léxico) de los Lenguajes

El análisis morfológico define la formación de los componentes léxicos básicos (tokens). Analizamos cómo manejan estos aspectos cuatro lenguajes clave:

Zig:

- * **Palabras reservadas:** Muy acotado (fn, pub, const, var, comptime, struct, defer).
- * **Identificadores:** Siguen el estándar alfanumérico. No admite redefinición de variables dentro del mismo bloque ni variables sin usar (provocan error de compilación).
- * **Literales:** Tipado estricto para enteros, flotantes y arreglos. Las cadenas son rebanadas de bytes constantes (`[]const u8`).
- * **Elementos irrelevantes:** Emplea llaves `{}` para agrupar bloques de código y punto y coma `;` para delimitar instrucciones. Los espacios y tabulaciones son ignorados. Comentarios inician con `//`.

Python:

- * **Palabras reservadas:** def, class, import, if, elif, else, while, for, lambda, entre otras.
- * **Identificadores:** Sensibles a mayúsculas y minúsculas; no pueden iniciar con números.
- * **Literales:** Dinámicos. Soporta cadenas con comillas simples o dobles, listas `[]`, tuplas `()`, diccionarios `{}` y conjuntos.
- * **Elementos irrelevantes:** Carece de llaves delimitadoras. Utiliza indentación significativa (espacios en blanco al inicio de la línea) para definir bloques sintácticos. Comentarios se marcan con `#`.

Rust:

- * **Palabras reservadas:** fn, let, mut, match, impl, struct, use, pub.
- * **Identificadores:** Convención snake_case para variables y funciones, y CamelCase para tipos/estructuras.
- * **Literales:** Rigurosamente tipados con sufijos opcionales (ej. 42u32, 3.14f64). Las cadenas pueden ser prestadas (&str) o dinámicas (String).
- * **Elementos irrelevantes:** Delimitado por llaves {} y requiere punto y coma ;. Espacios en blanco irrelevantes. Comentarios de línea con // y multilínea con /* ... */.

JavaScript (Node.js):

- * **Palabras reservadas:** const, let, var, function, class, return, async, await.
- * **Identificadores:** Convención típica alfanumérica, sensible a mayúsculas.
- * **Literales:** Dinámicos (números de punto flotante de doble precisión por defecto). Soporta cadenas con comillas simples, dobles o plantillas con backticks.
- * **Elementos irrelevantes:** Agrupación por llaves {}. El uso de punto y coma ; es opcional debido a la Inserción Automática de Puntos y Comas (ASI) del analizador léxico. Comentarios idénticos a los de C++/Java.

3.2. Análisis Sintáctico de Estructuras de Control

El análisis sintáctico valida el orden y la jerarquía de las instrucciones según la gramática del lenguaje:

- * **Python:** La sintaxis está dictada por dos puntos : y bloques indentados. No se requieren paréntesis en las condiciones (ej. if x > 5:). Las funciones se declaran con def.
- * **JavaScript:** La sintaxis es de estilo C. Las condiciones requieren paréntesis if (x > 5) {} y los bloques se delimitan con llaves. Las iteraciones soportan for, while y do-while.
- * **Rust:** Combina estilo de llaves sin requerir paréntesis en las condiciones (if x > 5 {}). Su análisis sintáctico es altamente robusto y trata casi todas las estructuras de control como expresiones que retornan valores (ej. let result = if x > 5 { 10 } else { 20 };). El emparejamiento con match es exhaustivo.
- * **Zig:** Sigue el enfoque de Rust/C. Las condiciones de bloque requieren paréntesis (if (x > 5) {}). Al igual que Rust, las estructuras como if y switch actúan como expresiones.

3.3. Arquitectura y Modelos de Ejecución Detallados por Lenguaje

Para comprender en su totalidad el comportamiento de estos lenguajes frente al compilador y en el entorno operativo, es necesario analizar de manera exhaustiva sus modelos de memoria y ciclos de ejecución:

3.3.1. Zig: Control de Memoria Explícito y Metaprogramación Estática

- * **Modelo de Memoria:** Zig elimina la asignación dinámica de memoria implícita u oculta. No posee recolector de basura ni un asignador global automático. Obliga a que las funciones que requieran memoria dinámica acepten un objeto alocador (`std.mem.Allocator`) explícitamente como parámetro, facilitando estrategias como `GeneralPurposeAllocator` (desarrollo y depuración) o `ArenaAllocator` (liberaciones en bloque).

- * **Compilación y Metaprogramación (comptime):** La metaprogramación se realiza en tiempo de compilación usando 'comptime', utilizando el propio lenguaje para ejecutar lógica y generar tipos antes de emitir el código de máquina final optimizado por LLVM.

- * **Manejo de Errores:** No implementa excepciones. Los errores son valores de un tipo enumerado especial combinados mediante un tipo unión (!T), forzando una gestión explícita y determinista sin saltos de pila inesperados.

3.3.2. Rust: Seguridad en Compilación sin Recolector de Basura

- * **Modelo de Memoria (Borrow Checker):** No requiere recolector de basura ni gestión manual. Implementa el modelo de Propiedad (Ownership), Préstamo (Borrowing) y Tiempos de Vida (Lifetimes) garantizados por el Borrow Checker, insertando liberaciones automáticas (drop) al salir del ámbito.

- * **Safe vs. Unsafe Rust:** Se divide estrictamente entre código seguro (garantías del compilador) y bloques unsafe (el desarrollador asume la responsabilidad del manejo de punteros crudos y APIs externas).

- * **Compilación:** El código se compila a una Representación Intermedia MIR, luego a LLVM IR y finalmente a código de máquina nativo optimizado.

3.3.3. Python (CPython): Máquina Virtual de Pila y el Global Interpreter Lock

* **Modelo de Ejecución (VM de CPython):** Compila a bytecode y ejecuta en una VM de pila. Cada variable es un PyObject en C con metadatos de tipos y contadores de referencias, lo que causa sobrecarga de memoria y CPU.

* **Gestión de Memoria y Recolector de Basura:** Combina conteo de referencias (liberación inmediata) con un recolector de basura generacional cíclico secundario para ciclos inaccesibles.

* **Global Interpreter Lock (GIL):** Cerrojo que impide ejecutar bytecode de Python simultáneamente en varios hilos nativos, bloqueando el paralelismo CPU-bound real y forzando multiprocesamiento.

3.3.4. JavaScript (V8 Engine): Compilación JIT de Doble Estapa y Concurrencia por Eventos

* **Compilación JIT (V8):** Usa dos etapas: Ignition (intérprete rápido de bytecode) y TurboFan (compilador optimizador JIT que compila funciones calientes a código de máquina basándose en retroalimentación de tipos, con des-optimización dinámica si fallan los tipos).

* **Gestión de Memoria:** Usa un GC generacional con un espacio joven (New Space - Scavenger) y un espacio viejo (Old Space - Mark-Sweep-Compact).

* **Modelo de Concurrencia (Event Loop y libuv):** JavaScript es monohilo. La concurrencia asíncrona se gestiona vía Event Loop y libuv, delegando I/O bloqueante al ThreadPool y procesando callbacks en el hilo principal de JS.

3.4. Resultados del Benchmarking (Algoritmo de Collatz)

Para evaluar el impacto de las tecnologías de compilación y ejecución, se diseñó un algoritmo intensivo en cómputo que calcula la longitud de la secuencia de la Conjetura de Collatz para cada número entero desde 1 hasta $N = 2.000.000$. A continuación se presentan los resultados obtenidos en el hardware local:

Lenguaje	Paradigma Dominante	Mecanismo Ejecución	Tiempo Prom.	Memoria Pico	Vel. Rel.
Python	Multiparadigma (POO, Imperativo)	Interpretado (CPython / VM)	151557.35 ms	0.0003 MB	1.0x
Java	Multiparadigma (Prototípico, Funcional)	Bytecode (JVM) / V8 Engine	2401.99 ms	40.9648 MB	63.1x
Rust	Multiparadigma (Funcional, Imperativo)	Compilación Nativa (LLVM)	1848.26 ms	1.2500 MB	82.0x
Zig	Imperativo / Estructurado	Compilación Nativa (LLVM)	1981.14 ms	1.1000 MB	76.5x

Análisis del Benchmark: Los resultados evidencian la enorme diferencia entre los lenguajes compilados nativamente (Rust y Zig), que completan la tarea en milisegundos gracias al diseño optimizado de bucles por LLVM, frente a Python, que sufre penalizaciones por la sobrecarga del bucle de evaluación e introspección de tipos en su máquina virtual.

4. Actividad III: Diseño de un Lenguaje de Dominio Específico (DSL)

Para solucionar de forma segura el control de la planta industrial crítica ECO-GRID (microrredes y almacenamiento de energía), se presenta el diseño formal del Lenguaje L.

4.1. Especificación del Alfabeto y Reglas Léxicas

- * **Identificadores:** [a-zA-Z_][a-zA-Z0-9_]*
- * **Literales Numéricos:** [0-9]+ (Enteros representando valores de potencia en kW o temperaturas en °C)
- * **Operador de Asignación:** :=
- * **Delimitador de Instrucción:** ;
- * **Delimitadores de Parámetros:** (y)
- * **Comentarios:** Delimitados por # al inicio de la línea.
- * **Ignorados:** Espacios en blanco, tabulaciones y saltos de línea (no significativos).

4.2. Palabras Clave Obligatorias

- * **init_grid:** Inicializa el driver de hardware de ECO-GRID.
- * **leer_temperatura(bateria_id):** Retorna la temperatura de la celda de batería en °C.
- * **estado_carga(bateria_id):** Retorna el porcentaje de carga 0-100%.
- * **conmutar_linea(sector_id, estado):** Actúa sobre los relés: 1 = conectar, 0 = aislar.
- * **si_verdadero ... entonces ... fin_si:** Estructura condicional.
- * **mientras ... ejecutar ... fin_mientras:** Estructura repetitiva.

4.3. Gramática Sintáctica Abstracta en EBNF

```

<programa> ::= "init_grid" ";" <sentencia>*
<sentencia> ::= <asignacion> | <condicional> | <bucle> | <llamada_accion> ";"
<asignacion> ::= <identificador> ":" <expresion> ";"
<condicional> ::= "si_verdadero" <comparacion> "entonces" <sentencia>* "fin_si"
<bucle> ::= "mientras" <comparacion> "ejecutar" <sentencia>* "fin_mientras"
<comparacion> ::= <expresion> <operador_rel> <expresion>
<operador_rel> ::= ">" | "<" | "==" | ">=" | "<=" | "!="
<expresion> ::= <identificador> | <numero> | <llamada_lectura>
<llamada_lectura> ::= "leer_temperatura" "(" <expresion> ")" | "estado_carga" "(" <expresion>
")"
<llamada_accion> ::= "conmutar_linea" "(" <expresion> "," <expresion> ")"
<numero> ::= [0-9]+

```

4.4. Escenario Operativo A: Prevención de Fuga Térmica

Este programa monitoriza la temperatura del banco de baterías 1 de forma continua. Si excede 55°C, se aísla térmicamente apagando la carga solar, activando ventiladores auxiliares y derivando la carga del sector industrial (línea 1) hacia la red comercial de respaldo (línea 2).

```

init_grid;
mientras 1 == 1 ejecutar
    temp := leer_temperatura(1);
    si_verdadero temp > 55 entonces
        conmutar_linea(5, 1); # Activar ventilador de refrigeración auxiliar (Línea 5)
        conmutar_linea(4, 0); # Desconectar arreglos de Paneles Solares (Línea 4) para
frenar carga
        conmutar_linea(1, 0); # Desconectar el sector industrial de las baterías (Línea 1)
        conmutar_linea(2, 1); # Conectar el sector industrial a la red de respaldo comercial
(Línea 2)
    fin_si
fin_mientras

```

4.5. Escenario Operativo B: Balance de Carga y Optimización Energética

Este script evalúa el estado de carga de las baterías. Si la carga supera el 90% y hay excedente solar, activa relés para vender electricidad. Si la carga cae por debajo del 20% durante la noche, apaga los sectores de consumo no esenciales para reservar energía en áreas críticas (médicas y servidores).

```
init_grid;
mientras 1 == 1 ejecutar
    carga := estado_carga(1);
    si_verdadero carga > 90 entonces
        # Carga óptima: inyectar energía a la red general pública (Línea 3)
        conmutar_linea(3, 1);
    fin_si

    si_verdadero carga < 20 entonces
        # Batería en nivel crítico: aislar zonas industriales no esenciales (Línea 1)
        # y mantener encendidos servidores y áreas médicas (Línea 6)
        conmutar_linea(1, 0);
        conmutar_linea(6, 1);
    fin_si
fin_mientras
```

5. Consideraciones sobre Inteligencia Artificial y Ética

El auge de la Inteligencia Artificial Generativa y los Grandes Modelos de Lenguaje (LLMs) ha redefinido las metodologías de enseñanza y el trabajo de ingeniería de software a escala mundial. En el ámbito académico de la UNEG, se establecen directrices claras sobre responsabilidad ética:

* **Uso como Habilitador Académico:** Se permite el uso de IA como un asistente de aprendizaje para depurar la sintaxis, documentar código y proponer optimizaciones en los algoritmos de benchmarking.

* **Responsabilidad Individual:** El uso de estas herramientas informáticas no exime de la autoría. El programador (el estudiante) asume la responsabilidad absoluta e individual sobre la validez, semántica y correcto funcionamiento del código entregado.

* **Comprensión Conceptual:** Cada estudiante debe poseer un dominio conceptual exhaustivo sobre toda la solución presentada. La evaluación individual (defensa) mide la capacidad de justificar técnicamente cada decisión de diseño del compilador frente al jurado evaluador, garantizando que el uso de IA sirva para potenciar el entendimiento y no para sustituir la capacidad cognitiva.

6. Conclusiones

1. El Lenguaje como Arquitectura: Los lenguajes de programación no son entes abstractos rígidos; sus reglas morfológicas e intérpretes determinan de forma fundamental el rendimiento del sistema final.

2. Desempeño Comparativo: El análisis de benchmarking evidencia que para sistemas críticos y de alto rendimiento, los lenguajes de compilación nativa (Rust y Zig) son indispensables debido a su mínima latencia y gestión de memoria de forma determinista y predecible.

3. Valor de los DSL: La creación de lenguajes de dominio específico como el Lenguaje L permite a los operadores de industrias críticas interactuar de forma segura con el hardware, limitando el espacio de errores lógicos.

7. Protocolo de Entrega y RETO DE FRASES

7.1. Protocolo de Entrega

De acuerdo con las instrucciones de la asignatura, la entrega de este proyecto debe ser realizada por el líder de grupo vía correo electrónico a la dirección oficial del docente, indicando:

1. Nombre del grupo y su eslogan.
2. Listado oficial de participantes.
3. Dirección del repositorio Git remoto conteniendo todos los códigos fuente organizados en directorios, el archivo README y el PDF del informe.
4. Enlace del video de la defensa en Google Drive (duración máxima de 10 minutos por participante).

7.2. RETO DE FRASES (Verificación de Lectura del PDF)

Como prueba inequívoca de la lectura exhaustiva y rigurosa del material oficial, se adjuntan a continuación las frases marcadas con el prefijo F: que se encontraban ocultas en el documento guía de la asignatura:

1. Frase 1 (Objetivos): 'F: aunque existen diferentes paradigmas siempre nos enamoramos de uno, pero esto no indica que no debemos dominar los demás y aplicarlo según las circunstancias.'
2. Frase 2 (Rust): 'F: Rust representa por su seguridad el nuevo estandar para la construccion de kernel Linux.'
3. Frase 3 (Análisis Léxico): 'F: La tokenización en lenguajes es formalmente diferente a la realizada para los LLM en procesamiento de lenguaje Natural.'
4. Frase 4 (Benchmarking): 'F: EL benchmarking es una herramienta comparativa interesante para ingeniero de software alto nivel.'
5. Frase 5 (Algoritmos): 'F: sería ideal hacer una gráfica del benchmarking.'
6. Frase 6 (DSL): 'F: Un ejercicio creativo para medir su nivel de abstracción en el diseño de un entorno físico con su respectiva comunicación o interfase hombre maquina (lenguaje), su imaginación es el limite!'
7. Frase 7 (Referencias): 'F: bajo las directrices de las Normas APA'

8. Referencias Bibliográficas

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). Compiladores: Principios, técnicas y herramientas (2da ed.). Pearson Educación.
- Ecma International. (2025). ECMAScript 2025 Language Specification. <https://tc39.es/ecma262/>
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2008). Introducción a la teoría de autómatas, lenguajes y computación (3ra ed.). Addison-Wesley.
- Python Software Foundation. (2026). The Python Language Reference (v3.12). <https://docs.python.org/3/reference/>
- Rust Project Developers. (2026). The Rust Reference. <https://doc.rust-lang.org/reference/>
- Zig Software Foundation. (2026). Zig Language Reference. <https://ziglang.org/documentation/>